# LLVM IR, Target Details, and SSA

## Compiler Engineering #2: From core IR objects to def-use based transformation

Gen Sakai

2026/07/03

B4, Taura Laboratory, The University of Tokyo

# Contents

1. Session Bridge . . . . . 2
2. Chapter 3: IR Objects and CFGs . . . . . 5
3. Chapter 7: LLVM IR and Targets . . . . . 11
4. Chapter 4: SSA and Optimization . . . . . 17
5. Hands-On: ir-ssa-lab . . . . . 22
6. Additional Exercises . . . . . 28

# 1. Session Bridge

# Where We Are

**Previous session**: set up LLVM, recapped the compiler pipeline, introduced ABI vocabulary and some jargons.

**Today**: LLVM IR. We will treat what IR actually looks like as an object graph, how control flow is represented, and how transformations stay correct. This session:

- inspects the program representation LLVM optimizes
- connects IR syntax to the C++ APIs that build and read it
- uses SSA def-use chains to perform a small, legality-checked transformation

# Textbook Map

| Theme | What we focus on |
|---|---|
| Chapter 3 | LLVM objects, CFGs, traversals, compiler concepts as APIs |
| Chapter 7 | IR syntax, types, target triple, data layout, attributes |
| Chapter 4 | SSA, dominance, def-use, legality of optimization |

These three build on each other: Chapter 3 gives the **structure** (objects and graphs), Chapter 7 gives **meaning** to what is inside each object (types, target details), and Chapter 4 gives the **rules** for changing that structure safely. The hands-on pass at the end touches all three in one place.

# 2. Chapter 3: IR Objects and CFGs

# Core Object Hierarchy

```
Module
  -> Function
       -> BasicBlock
            -> Instruction
```

```
define i32 @branch_and_phi(i32 %x, i1 %flag) {
entry:
  br i1 %flag, label %then, label %else
}
```

- `Module`: one translation unit – globals, function declarations/definitions, target triple, data layout
- `Function`: the `define ... @branch_and_phi(...) { ... }` block – owns a CFG and function-level attributes
- `BasicBlock`: the `entry:` label through its terminator – a straight-line instruction sequence that must end with a terminator (`br`, `ret`, `switch`); no ordinary instruction may follow one
- `Instruction`: each line inside a block, e.g. `br i1 %flag, ...` – a `Value` (can define a result) that is also a `User` (can consume other values as operands)

# From Theory to LLVM APIs

| Compiler concept | LLVM API surface |
|---|---|
| Translation unit | `llvm::Module` |
| Procedure | `llvm::Function` |
| CFG node | `llvm::BasicBlock` |
| Statement/value | `llvm::Instruction` |
| CFG successors | `successors(BB)` |
| CFG predecessors | `predecessors(BB)` |

```
for (BasicBlock *Succ : successors(BB)) { ... }
for (BasicBlock *Pred : predecessors(BB)) { ... }
```

A textbook's "walk the CFG successors" is this one-line loop in LLVM – the API mirrors the theory closely enough to translate pseudocode almost directly.

# CFG Traversal: Reverse Post-Order


A depth-first **post-order** visit finishes a node only after every node it reaches from there is finished. Here that finishing order is `exit, body, header, entry`. **Reverse** it: `entry, header, body, exit` – every predecessor now appears before its successors (the one exception is the backedge `body -> header`, which no linear order can satisfy).

*Forward dataflow analyses (reaching definitions, constant propagation) converge in fewer passes over the CFG in RPO, because a fact computed at `header` is already available before `body` is visited.*

# Edges That Matter

- **Backedge**: in a loop `header -> body -> ... -> header`, the edge back into `header` – points from inside a loop back to its header. Central to loop discovery.
- **Critical edge**: source has multiple successors **and** destination has multiple predecessors.

```
A -> B      A has two successors: B, C
A -> C      C has two predecessors: A, D
D -> C
```

  `A -> C` is critical: code inserted "on that edge alone" needs its own block, or it would also affect `A -> B` or `D -> C`.
- **Irreducible CFG**: a cycle with more than one entry from outside – no single block dominates the whole loop (next slide: a real one).

*Many transformations split a critical edge into a fresh block before inserting edge-specific code, precisely to avoid touching the other edges at A or C.*

# Irreducible CFG: A Real Example

```
int irreducible_loop(int shouldSkip1stCall) {
  int i = 0;
  if (shouldSkip1stCall) goto skip;
  do {
    someFct();
    skip:;
  } while (++i < 6);
  return 32;
}
```

Edges: `entry->do_body`, `entry->skip`, `do_body->skip`, `skip->do_body` (backedge), `skip->do_end`. The `goto` jumps straight into the loop body from outside, past `do_body`. So the cycle `{do_body, skip}` has **two** external entries.

```
[Block] %do_body preds=2 succs=1 idom=%entry
[Block] %skip    preds=2 succs=2 idom=%entry
```

*Neither block dominates the other – only `entry` does. In a normal loop, the header dominates the entire loop body; here no block plays that role.*

# 3. Chapter 7: LLVM IR and Targets

# IR Syntax

```
define i32 @branch_and_phi(i32 %x, i1 %flag) {
entry:
  br i1 %flag, label %then, label %else
then:
  %a = add nsw i32 40, 2
  br label %merge
else:
  %b = mul nsw i32 8, 2
  br label %merge
merge:
  %y = phi i32 [ %a, %then ], [ %b, %else ]
  ret i32 %y
}
```

- `@branch_and_phi`: a global identifier (functions and global variables use `@`)
- `%x`, `%flag`, `%a`, …: local SSA values (`%name`, or unnamed `%0`, `%1`, …)
- `entry:`, `then:`, …: basic block labels
- every instruction reads as `opcode type operands`: `add nsw i32 40, 2` is "add, no-signed-wrap, 32-bit integers, on 40 and 2"

*This textual form (`.ll`) and the in-memory object graph are two views of the same IR – `opt` parses one into the other.*

# Types

Single-value types:

- integers: `i1` (booleans, e.g. `%flag`), `i8`, `i32`, `i64`, even arbitrary widths like `i128`
- floating-point: `float`, `double`
- pointers: `ptr` (opaque since LLVM 15 – no pointee type baked into the type itself)
- vectors: `<4 x i32>` – one instruction acts on all lanes at once

Aggregate types:

- arrays: `[4 x i32]`
- structures: `{ i8, i64 }` (anonymous), or `%struct.Point = type { i32, i32 }` (named)

*Aggregate layout (padding, size) depends on the target ABI and data layout – next two slides.*

# Types in Practice

```
%struct.Point = type { i32, i32 }
%field = getelementptr inbounds %struct.Point, ptr %pts, i64 1, i32 1
%slot  = getelementptr inbounds [4 x i32], ptr %arr, i64 0, i64 2

%sum = add <4 x i32> %a, %b
```

- `%struct.Point`: a **named** struct type, vs. an anonymous struct type like `{ i8, i64 }` (no name, just a shape) – named types avoid repeating the same shape everywhere
- array GEP steps by index (`i64 2` = 3rd element); struct GEP steps by field (`i32 1` = 2nd field)
- `<4 x i32>`: one `add` instruction computes all four lane sums at once

# Target Triple and Data Layout

```
target triple = "x86_64-unknown-linux-gnu"
target datalayout = "e-m:e-p270:32:32-...-i64:64-...-n8:16:32:64-S128"
```

Triple = `arch-vendor-os-abi`: `x86_64` / `unknown` / `linux` / `gnu`.

Data layout tokens decoded:

- `e`: little-endian
- `i64:64`: `i64` naturally aligns to 64 bits
- `n8:16:32:64`: native integer widths the target's ALU handles directly
- `S128`: stack objects are naturally aligned to 128 bits

*The same IR can lower differently depending on these settings – IR is not fully target-independent.*

# Why Layout Changes IR Meaning

```
struct Pair { char tag; long value; };
```

- LP64 (Linux/macOS x86_64): `long` is 64 bits -> `tag` at offset 0, 7 bytes padding, `value` at offset 8, `sizeof` = 16
- LLP64 (Windows x86_64): `long` is 32 bits -> `tag` at offset 0, 3 bytes padding, `value` at offset 4, `sizeof` = 8

```
%field = getelementptr inbounds { i8, i64 }, ptr %p, i32 0, i32 1
```

*`getelementptr` computes this offset from the struct **type** plus the module's data layout – it is not a raw byte-pointer addition, so the same GEP instruction means different byte math on different ABIs.*

# 4. Chapter 4: SSA and Optimization

# SSA Values

```
then:
  %a = add nsw i32 40, 2
  br label %merge
else:
  %b = mul nsw i32 8, 2
  br label %merge
merge:
  %y = phi i32 [ %a, %then ], [ %b, %else ]
```

- each SSA value (`%a`, `%b`, `%y`) has **exactly one** definition in the whole function – there is no other `%a = ...` anywhere
- `phi` selects a value based on which predecessor control arrived from: `%y` is `%a` coming from `then`, `%b` coming from `else`
- a definition must dominate every one of its uses (next slide)

*SSA values are not source variables: `mem2reg` can split one C local variable into several SSA values, one per definition site.*

# Dominance

Block A **dominates** block B if every path from the function entry to B passes through A.

Using `branch_and_phi`'s CFG (`entry -> {then, else} -> merge`):

- `entry` dominates `then`, `else`, and `merge` – every path starts at `entry`
- `then` does **not** dominate `merge` – the `else -> merge` path never visits `then`
- so `%a` (defined in `then`) cannot be read directly in `merge`; that gap is exactly why `phi` exists

Dominance answers practical questions:

- is this definition available at this use?
- where must a `phi` be inserted?
- can this computation be hoisted earlier without becoming unavailable?

# Def-Use in LLVM

LLVM does not need a separate def-use table for ordinary IR – it is built into three interfaces:

- `Instruction` **is a** `Value` (it can be used, e.g. as `%a`)
- `Instruction` **is also a** `User` (it can use other values as operands)
- each operand slot is a `Use`

```
for (Use &U : I.operands())  // use-def: what I uses
for (User *U : I.users())    // def-use: who uses I's result
```

In `%y = phi i32 [ %a, %then ], [ %b, %else ]`: iterating `%a`'s `users()` finds the `phi`; iterating the `phi`'s `operands()` finds `%a` and `%b`.

`BO->replaceAllUsesWith(Folded)` *walks every current* `Use` *of* `BO` *and repoints it at* `Folded` *– this is how SSA rewriting stays consistent without a separate bookkeeping pass.*

# Legality Before Profitability

```
%bad = add nsw i32 2147483647, 1
```

This must not become `-2147483648`.

- `nsw` means "no signed wrap": the author asserted this addition will not signed-overflow
- if it does overflow anyway, the result is **poison**, not a wrapped-around number
- replacing poison with a concrete integer changes what the program means, not just what it computes

*The same care applies to `nuw`, `exact` division, and division by zero – see `evaluateConstantBinaryOp` in the hands-on pass.*

# 5. Hands-On: ir-ssa-lab

# Your Task

`src/IRAndSSALabPass.cpp` is the finished pass – the answer key. Your job is to fill in the three TODOs in `template/IRAndSSALabPass.cpp` (built separately as `IRAndSSALabPassTemplate`, so `src/` stays untouched):

1. `printFunctionIRFacts`: print function/block counts, the RPO order, each block's predecessor/successor counts and immediate dominator, and per-instruction opcode/type/operands/users/phi-incoming
2. `evaluateConstantBinaryOp`: fold `Add` / `Sub` / `Mul` / `UDiv` / `SDiv` / `And` / `Or` / `Xor` with `APInt`, respecting `nsw` / `nuw` / `exact`, and rejecting division by zero and signed `INT_MIN / -1`
3. `foldConstantBinaryOperators`: drive the fold with a worklist, `replaceAllUsesWith`, and safe dead-instruction erasure

This one pass is where Chapter 3, 7, and 4 all come together – object/CFG facts, target info, and a legality-checked rewrite in one place.

*Do not tackle this head-on: do `exercises/01_const_fold_standalone` first (items 2/3, no printing/RPO/dominance yet), then fill in item 1 one stage at a time. Each stage's TODO comment includes a literal code hint – type it in rather than reasoning out the API from scratch. `tests/template_checkpoints.ll` has `STEP1` / `STEP2` / `STEP3` checks for feedback before you reach the fold.*

# What the Pass Prints

Running with `-disable-output` on `tests/test.ll`:

```
[Module] name=tests/test.ll
[Target] triple=x86_64-unknown-linux-gnu
[Function] @branch_and_phi args=2 blocks=4
[CFG-RPO] %entry %then %else %merge
[Block] %merge preds=2 succs=0 idom=%entry
  [Inst] %y opcode=phi type=i32
    operands: 42 16
    users: %z
    phi-incoming: [42 from %then] [16 from %else]
```

Every line traces back to a concept from this deck: `[Module]` / `[Target]` (Chapter 7), `[Function]` / `[Block]` / `[Inst]` (Chapter 3), `idom` / `phi-incoming` / `users` (Chapter 4).

*This and the next slide are your target: check your* `template/` *work by pointing* `opt` *at* `build/IRAndSSALabPassTemplate.dylib` *instead of* `build/IRAndSSALabPass.dylib` *and comparing.*

# What the Pass Transforms

```
%a = add nsw i32 40, 2
%b = mul nsw i32 8, 2
%y = phi i32 [ %a, %then ], [ %b, %else ]
```

becomes:

```
%y = phi i32 [ 42, %then ], [ 16, %else ]
```

- `%a` / `%b` are not moved into the `phi` – every **use** of `%a` / `%b` is repointed to the folded constant via `replaceAllUsesWith`
- `%a` / `%b` are then erased: they have no users left, and `isInstructionTriviallyDead` confirms `add` / `mul` have no side effects

# Implementation Hooks

```
FunctionAnalysisManager &FAM =
    MAM.getResult<FunctionAnalysisManagerModuleProxy>(M).getManager();

DominatorTree &DT = FAM.getResult<DominatorTreeAnalysis>(F);

for (BasicBlock *BB : ReversePostOrderTraversal<Function *>(&F))
  ...
```

- the pass is a **module** pass (it needs `Module`-level target info), but dominance is a **function**-level analysis – the proxy line bridges the two managers
- `DominatorTreeAnalysis` computes the whole dominator tree once per function; `getNode(BB)->getIDom()` then reads off one block's immediate dominator
- `ReversePostOrderTraversal` is the same RPO from the "CFG Traversal" slide, ready to iterate

*Everything above maps directly to a slide you have already seen – that is the point of this pass.*

# Run Commands

```
cd 02_ir_and_ssa
cmake -S . -B build -DLLVM_DIR="$(llvm-config --cmakedir)"
cmake --build build

opt -load-pass-plugin=build/IRAndSSALabPass.dylib \
  -passes="ir-ssa-lab" \
  -disable-output tests/test.ll
```

- `-disable-output`: run the pass for its printed trace only, discard the transformed IR
- drop `-disable-output` and add `-S` instead to print the transformed IR

*On Linux, the plugin extension is usually `.so` instead of `.dylib`.*

# 6. Additional Exercises

# Three More Ways In

Each is a plain executable – no plugin, no `opt -load-pass-plugin`. All follow a `your_turn/` (TODO) vs. `solution/` split with a `main.cpp` harness that compares both.

1. `exercises/01_const_fold_standalone` (**before** the main lab): fold, respecting `nsw`, one function at a time – no RPO/dominance/printing yet
2. `exercises/02_build_module_irbuilder` (**any time after Chapter 3**): build `branch_and_phi` from scratch with `IRBuilder`
3. `exercises/03_def_use_scope_change` (**alongside Def-Use in LLVM**): a demo, not a fill-in exercise – def-use chains cross function boundaries

# IRBuilder: Building What We've Been Reading

```
IRBuilder<NoFolder> Builder(ThenBB);
Value *A = Builder.CreateNSWAdd(
    ConstantInt::get(Int32Ty, 40), ConstantInt::get(Int32Ty, 2), "a");

Builder.SetInsertPoint(MergeBB);
PHINode *Y = Builder.CreatePHI(Int32Ty, 2, "y");
Y->addIncoming(A, ThenBB);
```

This builds exactly the `branch_and_phi` function shown throughout this deck: `IRBuilder` places one instruction at a time at its current insertion point, and `PHINode::addIncoming` records each `[value, predecessor]` pair explicitly – there is no other way to build a `phi`.

*Plain `IRBuilder<>` would constant-fold `40 + 2` immediately, so `%a` would never become a real instruction – `NoFolder` keeps this exercise literal.*

# Def-Use Crosses Function Boundaries

```
@counter = external global i32

define i32 @read_in_foo() {
  %v = load i32, ptr @counter
  ...
}
define i32 @read_in_bar() {
  %v = load i32, ptr @counter
  ...
}
```

`@counter` is a `Value` like any other. Walking its use-list from the `load` inside `@read_in_bar` also finds the `load` inside `@read_in_foo` – a def-use edge that crosses a function boundary.

*Dominance is scoped to a function (it is a per-`Function` analysis). Def-use is not: any `User` anywhere in the `Module` can hold a `Use` of a `GlobalVariable`.*

# Check Your Understanding (1/2)

- **Which LLVM object corresponds to each line of IR?** `Module` = whole file; `Function` = `define ... { ... }`; `BasicBlock` = a label through its terminator; `Instruction` = each line inside a block.
- **Why does the phi node live in `merge`?** Because that is where the `then` / `else` paths reunite – a phi only makes sense at a block with multiple predecessors.
- **Which values dominate `%z` in `branch_and_phi`?** `%x`, `%flag` (defined in `entry`, which dominates everything) and `%y` (defined in `merge`, the block `%z` is in).
- **Why can `%a` / `%b` be erased after replacement?** `replaceAllUsesWith` leaves them with zero users, and `add` / `mul` have no side effects, so `isInstructionTriviallyDead` allows it.

# Check Your Understanding (2/2)

- **Why does the `nsw` overflow example remain unchanged?** Folding it would replace a poison value with a concrete number, changing what the program means.
- **In `irreducible_loop`, why isn't `do_body` or `skip` a valid loop header?** A header must dominate the whole loop body. Here `entry` can reach either block by a path that skips the other, so neither dominates the other.
- **If an `alloca` replaced the global in the def-use demo, could def-use still cross functions?** No – an `alloca`'s address never reaches another function unless it is explicitly passed as an argument, so nothing outside the defining function can hold a `Use` of it.